

Kinekt: Local-First Developer Context Engine

Architecture & Technical Design Document

Author:	Lead Data Platform & AI Engineer	Status:	Approved for Implementation
Version:	1.0.0	Date:	June 12, 2026
Target Stack:	Python, FastMCP, LangGraph, ChromaDB	Deployment:	Localhost (Zero-Cloud Dependency)

1. Executive Summary & Goals

Modern software engineers face severe cognitive overhead due to context-switching between fragmented repositories, local git changes, and unstructured personal knowledge bases (such as Obsidian, Notion exports, or local markdown files).

Kinekt resolves this problem by acting as a highly optimized, local-first data and orchestration system. It exposes a developer's local machine resources safely and standardizes access using Anthropic's **Model Context Protocol (MCP)**. Powered by **LangGraph** for advanced stateful reasoning and **ChromaDB** for high-performance localized Retrieval-Augmented Generation (RAG), Kinekt transforms individual dev workspaces into an intelligent, queryable graph network.

2. Architectural Boundary & Project Scope

In-Scope (MVP Phase)

- **Strict Local Isolation:** 100% data privacy. Local parsing, local vector embedding pipelines, and local LLM execution via Ollama.
- **Multi-Source Data Ingestion:** Abstract Syntax Tree (AST) code splitting coupled with semantic chunking for markdown prose.
- **Real-Time Workspace Tooling:** Custom MCP tools that read live git configurations, track file mutations, and parse unstaged code diffs.
- **Stateful Session Graph:** Multi-turn agent loops capable of managing short-term conversational context and running recursive self-correction strategies.

Out-of-Scope (Future Enhancements)

- Cloud-synchronized multi-tenant vector storage.
- Automated write operations back to source control (the tool generates recommendations and file payloads but explicitly delegates file writes and commits to the engineer for safety boundaries).

3. System Architecture & Component Design

The system is heavily decoupled to enforce clean separation of concerns, aligning precisely with advanced data engineering and AI system design patterns:

Layer	Core Technology	Responsibilities
Orchestration Layer	LangGraph, Ollama (Llama 3.1)	Manages agentic loops, controls system states, handles multi-turn thread memory, and routes queries.
Transport & Tool Layer	FastMCP Python SDK	Exposes secure JSON-RPC interfaces over standard I/O; encapsulates local file and system primitives into standard schemas.
Data & Storage Layer	ChromaDB, SQLite, Tree-sitter	Handles AST code chunking, maintains document embedding matrices, stores relational file registry states and thread history checkpoints.

4. Technical Stack & Dependency Inventory

- **Core Engine Runtime:** `Python 3.11+` for robust pipeline processing and native AI framework compatibility.
- **Agent Workflow Framework:** `langgraph` (v0.2+) for constructing cyclic, graph-based agent topologies.
- **Protocol Interface:** `mcp` via Python FastMCP framework to abstract tool definitions.
- **Vector Storage & Embeddings:** `chromadb` matching local `ollama` configurations utilizing the `V_{embed}` model `nommic-embed-text`.
- **Relational Persistence:** `SQLAlchemy` with `SQLite` backends for handling low-latency ACID operations.
- **Syntactical Extraction:** `tree-sitter` for syntax tree navigation across target coding languages.

5. Schema Architecture & Data Contracts

5.1 Relational Database Engine (SQLite)

```
-- Tracks interactive developer loops
CREATE TABLE sessions (
  session_id TEXT PRIMARY KEY,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Internal state saver for LangGraph execution threads
CREATE TABLE thread_messages (
  message_id TEXT PRIMARY KEY,
  session_id TEXT REFERENCES sessions(session_id),
  role TEXT CHECK(role IN ('user', 'assistant', 'tool')),
  content TEXT,
  timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Delta tracking engine to prevent duplicate vector encoding passes
CREATE TABLE file_registry (
  file_path TEXT PRIMARY KEY,
  file_type TEXT CHECK(file_type IN ('code', 'markdown')),
  last_modified_hash TEXT,
  last_indexed_at TIMESTAMP
);

-- Long-term persistent developer profiles (across multiple session lifetimes)
CREATE TABLE developer_profile (
  key TEXT PRIMARY KEY,
  value TEXT,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

5.2 Vector Space Vector Collection (ChromaDB)

Collection 1: `collection_code_chunks`

- **Embedding Matrix:** 768-dimensional float vectors generated via Nomic Embed text.
- **Metadata Structural Contract:** `{"file_path": str, "language": str, "construct_type": "function" | "class" | "module"}`

Collection 2: `collection_notes_chunks`

- **Embedding Matrix:** 768-dimensional float vectors.

- **Metadata Structural Contract:** `{"file_path": str, "tags": str, "heading_context": str}`

6. Implementation & Phased Milestone Plan

Milestone 1: The Local Unstructured Data Pipeline (Data Engineering Focus)

- Establish structural workspace scaffolding managed via modern dependency managers (e.g., `uv` or `poetry`).
- Develop the `data_pipeline/parser.py` component incorporating specialized Tree-sitter configurations to cleanly extract and break up code structures by local boundaries rather than standard character chunk intervals.
- Connect the embedding generation workflows to local Ollama endpoints and establish basic semantic query patterns against local ChromaDB instances.

Milestone 2: The FastMCP Protocol Gateway (Infrastructure Focus)

- Incorporate the FastMCP Python layer to construct a localized, low-latency background microservice.
- Expose and register explicit system tools: `query_knowledge_base`, `get_git_context`, and `read_workspace_file`.
- Validate execution of JSON-RPC protocol messages over local standard input/output pipelines.

Milestone 3: LangGraph Agent & Persistence Core (AI Engineering Focus)

- Design state topologies mapping out agent decision nodes, tool validation patterns, and structured routing frameworks.
- Implement the native LangGraph SQLite checkpointer backend to safely capture, save, and reload complete execution states across distinct developer tasks.
- Tune and calibrate base system prompt architectures to guarantee reliable multi-tool invocation paths when paired with quantized local models (e.g., `llama3.1:8b`).

Milestone 4: Presentation & Evaluation Layer

- Implement a clean, local UI using Streamlit or custom Next.js configurations hooked directly to standard Python FastAPI routes.
- Expose visual streaming capabilities detailing token emission data alongside active logs detailing what system tools are being invoked behind the scenes.
- Enable public LangSmith trace exports inside repository documentation blocks to showcase processing pipelines to engineering leaders and recruiters seamlessly.